

01 Exercise

All = {1,2,3,4,5,6,7,8,9,10}

Even = {2,4,6,8,10}

Odd = {1,3,5,7,9}

Even U Odd: (1,2,3,4,5,6,7,8,9,10)

Even n Odd: ()

All - Odd: (2,4,6,8,10)

C(Even): (1,3,5,7,9)

C(Odd-All): (1,2,3,4,5,6,7,8,9,10)

02 Exercise JOINS

Investigue y busque documentación sobre el SELECT que explique cómo utilizar las funcionalidades de:

SELECT

Is the most commonly used statement in SQL. It is used to retrieve data stored in database tables. With SELECT, it is possible to return specific columns, apply expressions and calculations, filter data, and even execute queries without referencing a table in some database systems.

ORDER BY

The ORDER BY clause is used to sort the results returned by a SELECT statement. Data can be sorted in ascending order (ASC) or descending order (DESC).

LIMIT

The LIMIT clause restricts the number of rows returned by a query. It is especially useful when working with large tables and only a sample of the data is needed.

GROUP BY

The GROUP BY clause groups rows that share the same value in one or more columns. It is commonly used together with aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX().

Without GROUP BY, aggregate functions operate on all rows in the table. With GROUP BY, calculations are performed for each group separately.

JOIN

A JOIN is used to combine data from two or more tables based on a relationship between them. Joins allow information stored in separate tables to be retrieved in a single query.

INNER JOIN

INNER JOIN is the primary tool used to combine data from two or more tables. It returns only the rows that have matching values in both tables.

LEFT JOIN

LEFT JOIN, also known as LEFT OUTER JOIN, returns all rows from the left table, whether or not there is a matching record in the right table. If no match exists, the columns from the right table contain NULL values.

RIGHT JOIN

RIGHT JOIN, also known as RIGHT OUTER JOIN, is the opposite of LEFT JOIN. It returns all rows from the right table, whether or not there is a matching record in the left table. If no match exists, the columns from the left table contain NULL values.

Informational Table

Funcionality	Description	Basic Syntax	Main Use
SELECT	Allows retrieving data stored in one or more database tables.	SELECT columna1, columna2 FROM tabla;	Query information stored in a database.
ORDER BY	Sorts the results returned by a query.	SELECT * FROM tabla ORDER BY columna ASC;	Arrange data in ascending or descending order.
LIMIT	Restricts the number of rows returned by a query.	SELECT * FROM tabla LIMIT 5;	Display only a specific number of records.
GROUP BY	Groups rows that share the same value and allows the use of aggregate functions.	SELECT columna, COUNT(*) FROM tabla GROUP BY columna;	Group data and perform calculations such as COUNT, SUM, or AVG.
INNER JOIN	Returns only the records that have matching values in both tables.	SELECT * FROM tabla1 INNER JOIN tabla2 ON tabla1.id = tabla2.id;	Combine related records from multiple tables.
LEFT JOIN	Returns all records from the left table and the matching records from the right table.	SELECT * FROM tabla1 LEFT JOIN tabla2 ON tabla1.id = tabla2.id;	Keep all records from the primary table, even if no match exists in the second table.
RIGHT JOIN	Returns all records from the right table and the matching records from the left table.	SELECT * FROM tabla1 RIGHT JOIN tabla2 ON tabla1.id = tabla2.id;	Keep all records from the secondary table, even if no match exists in the first table.

Inserts

```
1 INSERT INTO AUTHORS (ID, NAME) VALUES
2 (1, 'Miguel de Cervantes'),
3 (2, 'Dante Alighieri'),
4 (3, 'Takehiko Inoue'), "Takehiko": Unknown word.
5 (4, 'Akira Toriyama'), "Akira": Unknown word.
6 (5, 'Walt Disney');
```

```
1 INSERT INTO BOOKS (ID, NAME, ID_AUTHOR) VALUES
2 (1, 'Don Quijote', 1), "Quijote": Unknown word.
3 (2, 'La Divina Comedia', 2), "Divina": Unknown word.
4 (3, 'Vagabond 1-3', 3),
5 (4, 'Dragon Ball 1', 4),
6 (5, 'The Book of the 5 Rings', NULL);
```

```
1 INSERT INTO CUSTOMERS (ID, NAME, EMAIL) VALUES
2 (1, 'John Doe', 'j.doe@email.com'),
3 (2, 'Jane Doe', 'jane@doe.com'),
4 (3, 'Luke Skywalker', 'darth.son@email.com'); "Skywalker": Unknown word.
```

```
INSERT INTO RENTS (ID, ID_BOOK, ID_CUSTOMER, STATE) VALUES
(1, 1, 2, 'Returned'),
(2, 2, 2, 'Returned'),
(3, 1, 1, 'On time'),
(4, 3, 1, 'On time'),
(5, 2, 2, 'Overdue');
```

```
SQL > < 1 / 1 > 1 - 0 of 0
INSERT INTO AUTHORS (ID, NAME) VALUES
(1, 'Miguel de Cervantes'),
(2, 'Dante Alighieri'),
(3, 'Takehiko Inoue'),
(4, 'Akira Toriyama'),
(5, 'Walt Disney');
```

```
SQL > < 1 / 1 > 1 - 0 of 0
INSERT INTO BOOKS (ID, NAME, ID_AUTHOR) VALUES
(1, 'Don Quijote', 1),
(2, 'La Divina Comedia', 2),
(3, 'Vagabond 1-3', 3),
(4, 'Dragon Ball 1', 4),
(5, 'The Book of the 5 Rings', NULL);
```

```
SQL > < 1 / 1 > 1 - 0 of 0
INSERT INTO CUSTOMERS (ID, NAME, EMAIL) VALUES
(1, 'John Doe', 'j.doe@email.com'),
(2, 'Jane Doe', 'jane@doe.com'),
(3, 'Luke Skywalker', 'darth.son@email.com');
```

```
SQL > < 1 / 1 > 1 - 0 of 0
INSERT INTO RENTS (ID, ID_BOOK, ID_CUSTOMER, STATE) VALUES
(1, 1, 2, 'Returned'),
(2, 2, 2, 'Returned'),
(3, 1, 1, 'On time'),
(4, 3, 1, 'On time'),
(5, 2, 2, 'Overdue');
```

Consultas

1- Obtenga todos los libros y sus autores (en caso de tenerlos)

```
SELECT
BOOKS.NAME AS BOOK_NAME,
AUTHORS.NAME AS AUTHOR_NAME
FROM BOOKS
LEFT JOIN AUTHORS
ON AUTHORS.ID = BOOKS.ID_AUTHOR;
```

```
SQL > < 1 / 1 > 1 - 5 of 5
```

BOOK_NAME	AUTHOR_NAME
Don Quijote	Miguel de Cervantes
La Divina Comedia	Dante Alighieri
Vagabond 1-3	Takehiko Inoue
Dragon Ball 1	Akira Toriyama
The Book of the 5 Rings	NULL

2- Obtenga todos los libros que no tienen autor

```
SELECT NAME FROM BOOKS WHERE ID_AUTHOR IS NULL;
```

```
SQL > < 1 / 1 > 1 - 1 of 1
```

NAME
The Book of the 5 Rings

3- Obtenga todos los autores que no tienen libros

```
SELECT AUTHORS.NAME
FROM AUTHORS
LEFT JOIN BOOKS
ON AUTHORS.ID = BOOKS.ID_AUTHOR
WHERE BOOKS.ID IS NULL;
```

```
SQL > < 1 / 1 > 1 - 1 of 1
```

NAME
Walt Disney

4- Obtenga todos los libros que han sido rentados en algún momento

```
SELECT DISTINCT BOOKS.NAME
FROM BOOKS
INNER JOIN RENTS
ON BOOKS.ID = RENTS.ID_BOOK;
```

SQL ▾ < 1 / 1 > 1 - 3 of 3

NAME
Don Quijote
La Divina Comedia
Vagabond 1-3

5- Obtenga todos los libros que nunca han sido rentados

```
SELECT BOOKS.NAME
FROM BOOKS
LEFT JOIN RENTS
ON BOOKS.ID = RENTS.ID_BOOK
WHERE RENTS.ID IS NULL;
```

SQL ▾ < 1 / 1 > 1 - 2 of 2

NAME
Dragon Ball 1
The Book of the 5 Rings

6- Obtenga todos los clientes que nunca han rentado un libro

```
SELECT CUSTOMERS.NAME
FROM CUSTOMERS
LEFT JOIN RENTS
ON CUSTOMERS.ID = RENTS.ID_CUSTOMER
WHERE RENTS.ID IS NULL;
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

NAME
Luke Skywalker

7- Obtenga todos los libros que han sido rentados y están en estado “Overdue”

```
SELECT DISTINCT BOOKS.NAME
FROM BOOKS
INNER JOIN RENTS
ON BOOKS.ID = RENTS.ID_BOOK
WHERE RENTS.STATE = 'Overdue';
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

NAME
La Divina Comedia